

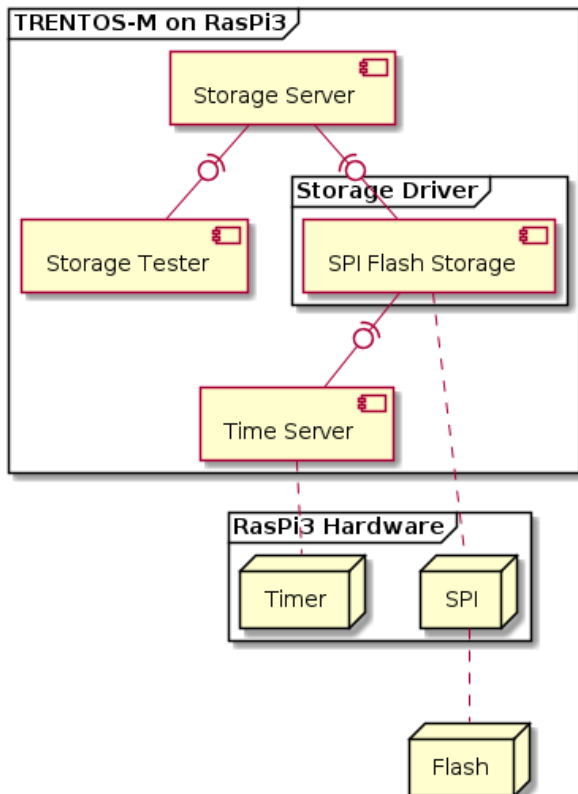
TRENTOS-M Flash test

- [General Information](#)
- [CAmkES Component Architecture](#)
- [Hardware Setup](#)
- [Implementation Details](#)
 - [Detecting memory size](#)
 - [Testing every single block of memory](#)
 - [Performance Tweaking](#)
- [Test scenario](#)
- [Prerequisites](#)
 - [Install sd-muxer application](#)
 - [Install docker](#)
 - [Cloning TRENTOS-M repo](#)
 - [Installing necessary boot files for Raspberry Pi](#)
- [Building the test application](#)
 - [Copy the os_image.bin onto the SD card](#)
- [Run the test](#)
- [Building and running demo_hello_world](#)

General Information

In the current version of TRENTOS-M, there is only support for SPI-based NOR-Flash for the Raspberry Pi. These NOR-Flash chips are bought in mass and may contain defect chips. For the IoT demo, 8 MiB flashes were bought. In order to differentiate which of these flashes are working and which are defect, this test application was developed. It will first detect the actual size of storage on the flash chip. In case the detected memory size is actually 8MiB, every single block of memory is tested (pattern detection and erase block). In the end, the test application tells the user if the flash is okay or defect.

CAmkES Component Architecture



Flash test for Raspberry Pi - CAmkES Architecture

Hardware Setup

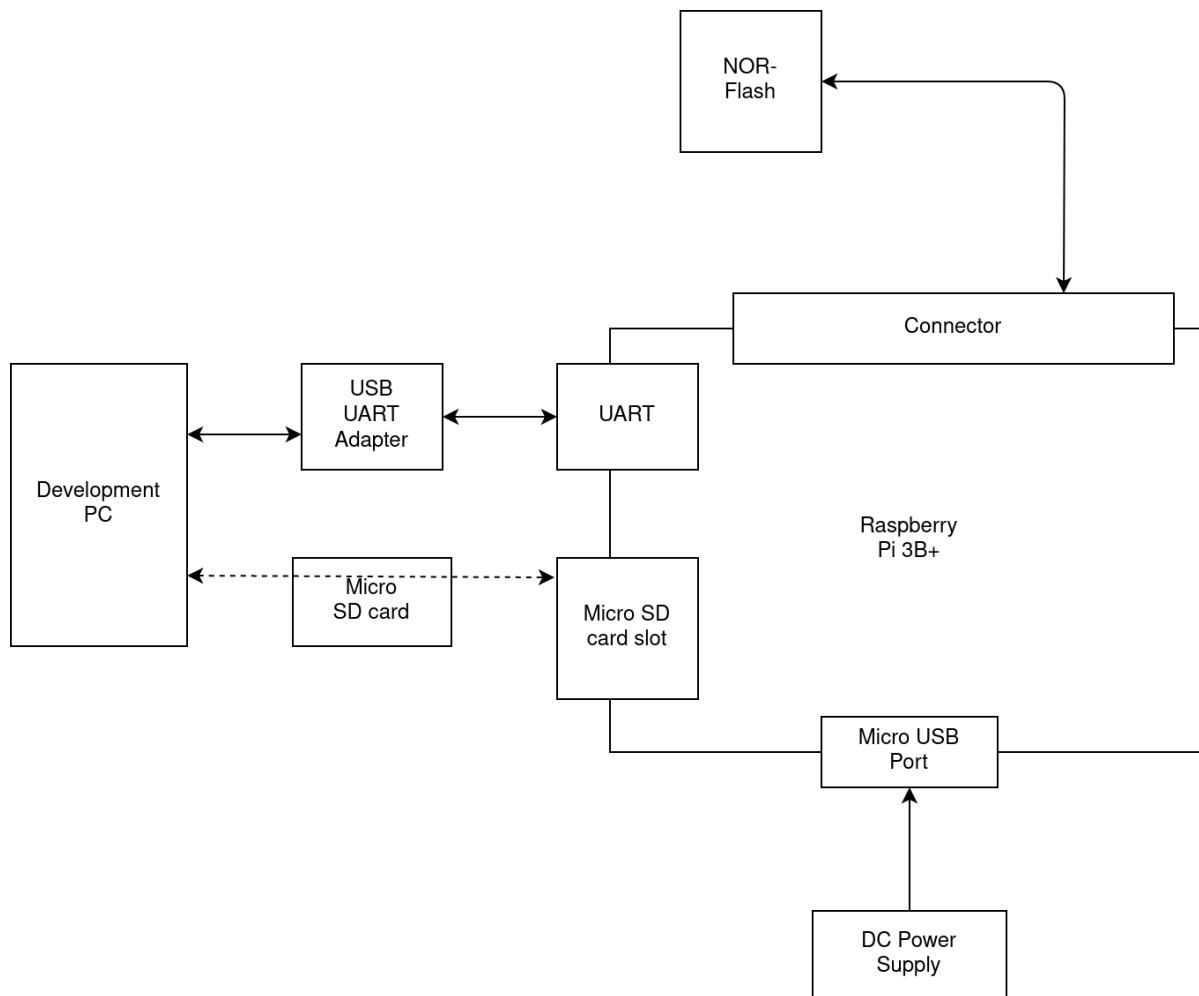
This demo is supposed to run standalone on the Raspberry Pi 3 Model B+.

In order to run the demo, the following is required:

- Raspberry Pi 3 Model B+ incl. power supply
- UART-to-USB adapter (see [RPI boot process](#), section "Boot up" for correct wiring) in order to observe RPi behaviour

- MicroSD Card that contains the flash test application
- NOR-Flash that is connected via SPI to Raspberry Pi

The following diagram shows the hardware setup:



SD Demo for Raspberry Pi - HW Setup

Implementation Details

Detecting memory size

The idea behind memory size detection is to write to an address that is one block larger than the actual memory size of the flash chip. This would lead to a write operation on the very first block of the flash chip as flash memory wraps around at the end of the available storage. An overflow is detected if the first block of the flash is different than before the write operation. The idea would be to write a certain pattern to the first memory block of the flash. Afterwards, increasing memory sizes are written with yet another bit pattern. After every write operation, the bit pattern of the first block is tested again. In case an overflow occurred, the original bit pattern of the first block is overwritten. Thus, the actual size of the flash chip is detected.

As flash chips usually come in sizes to the power of 2, we can double the suspected memory size on every memory test operation up to 8 MiB of memory starting from a block size of 4096 Bytes. This reduces the number of actual tests to detect the available size to $\log_2(\text{FLASH_SZ}/\text{BLOCK_SZ})$.

Testing every single block of memory

This sub-test validates every single block of memory. First a bit pattern is validated for a particular block, then the memory block is erased.

Performance Tweaking

When using the default settings for the flash configuration, the test performs very poorly. For a 8MiB flash, the test runs for about 24 minutes. As the settings in the default flash configuration, set the write and erase times to the maximal values of the datasheet (write: 15ms, erase: 300ms), these values can be tweaked. Especially the erase time can be greatly reduced. When set to the minimal times (write: 10ms, erase: 45ms), the test duration decreases to only 6 minutes. Especially when testing multiple of these flashes, these time savings greatly reduce overall testing time. The new timing values are implemented on branch SEOS-1774 in "seos_sandbox/components/RPI_SPI_FLASH".

It should be mentioned that the maximal values were set due to the fact that the maximal values are guaranteed times. When setting to minimal values, it depends on how fast the internal queue of the flash is filled and might lead to issues. To guarantee these minimal values busy flag polling should be implemented in the flash driver.

Test scenario

The idea behind the test application is minimal user intervention and should work in a plug-and-play like fashion. The tester has a box of multiple of these flashes and needs to find out which of these flash chips are working and which are defect. The tester takes a flash out of the box, connects it to the Raspberry Pi over SPI and turns the power on. The test application runs through and detects if the flash chip is working or defect. In case the flash chip is defect, it is sorted out. The Raspberry Pi is turned of again and the whole process starts again.

Prerequisites

Install sd-muxer application

- Clone and install application from: <https://github.com/xypron/sd-mux>
- Install required packages:

Required packages for sd-mux

```
sudo apt install libftdi1-devsudo apt install libpopt-devsudo apt install cmake
```

- Clone, build and install application:

clone,build and install sd-mux application

```
#clone repogit clone https://github.com/xypron/sd-mux.git#build sd-mux applicationcd sd-muxmkdir buildcd buildcmake ..make#install sd-mux applicationsudo make install
```

- find out serial ID of sd-mux usb connection:

find out serial number

```
sudo sd-mux-ctrl -l
```

- copy serial number and use the following commands to switch between laptop and board:

switch sd-card

```
# switch to laptopsudo sd-mux-ctrl -e <serial_id> -s# switch to boardsudo sd-mux-ctrl -e <serial_id> -d
```

- For more information on sd-muxer, consult the following: [SD-Card Switcher](#)

Install docker

- install docker according to <https://docs.docker.com/engine/install/ubuntu/>:

install packages

```
sudo apt-get updatesudo apt-get install \    apt-transport-https \    ca-certificates \    curl \    gnupg-agent \    software-properties-commoncurl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -sudo apt-key fingerprint 0EBFCD88sudo add-apt-repository \    "deb [arch=amd64] https://download.docker.com/linux/ubuntu \    $(lsb_release -cs) \    stable"sudo apt-get updatesudo apt-get install docker-ce docker-ce-cli containerd.io
```

- Get test and build images:

get test and build container

```
docker pull docker:5000/trentos_testdocker pull docker:5000/trentos_build
```

Cloning TRENTOS-M repo

- clone seos_tests repo from Bitbucket:

install seos_tests

```
git clone ssh://git@bitbucket.hensoldt-cyber.systems:7999/hc/seos_tests.gitgit checkout SEOS-1774cd seos_sandbox/components/RPI_SPI_FLASHgit checkout SEOS-1774
```

Installing necessary boot files for Raspberry Pi

- copy boot files to SD card of raspberry pi:

necessary boot files for raspi

```
cp sdk/resources/rpi3_sd_card/* <SD_Card_Mount_Dir>/
```

Building the test application

First you need to check out branch SEOS-1774 in "seos_sandbox/components/RPI_SPI_FLASH" for the other timing values to take effect:

Checkout SEOS-1774 branch

```
cd seos_sandbox/components/RPI_SPI_FLASHgit checkout "SEOS-1774"
```

For building the flash test application, the **build-system.sh** script has to be used and executed within the **trentos_build** docker container. The following command will invoke this custom build script from inside the build docker container. The container will bind the current working folder to a volume mounted under "/host", execute the script and then self remove.

Build test application

```
seos_sandbox/scripts/open_trentos_build_env.sh seos_sandbox/build-system.sh src/tests/test_storage rpi3 build-rpi3-Debug-test_storage -DCMAKE_BUILD_TYPE=Debug
```

As a result, the folder **build-rpi3-Debug-test_storage** is created, containing all the build artifacts. The TRENTOS-M system image can be found in **build-rpi3-Debug-test_storage/images/os_image.bin**.

Please also refer to the "[Getting Started](#)" guide for more helpful information on how to build TRENTOS-M systems.

Copy the os_image.bin onto the SD card

Get <sd_device_number> with:

SD Card switcher - Get SD device number

```
sudo sd-mux-ctrl -l
```

Get <block_device_number> of SD card:

Get block device number of inserted SD card

```
lsblk
```

Copy *os_image.bin* to Raspberry Pi with SD-card switcher:

copy os_image.bin to SD card with SD card switcher

```
sudo sd-mux-ctrl -e <sd_device_number> -ssudo mount /dev/<block_device_name> /mntcd <path_to_trentos_dir>
/build-rpi3-Debug-test_storage/imagessudo cp os_image.bin /mntsyncsudo umount /mntsudo sd-mux-ctrl -e
<sd_device_number> -d
```

Run the test

Approximate test duration (for one flash): ~6 minutes

To run the test application, you need to have the same setup as depicted in the "Hardware Setup" section. You also need serial access to the output of the Raspberry Pi over UART with the following command:

Get serial output

```
sudo picocom -b 115200 /dev/ttyUSB0
```

If this is the case, you can execute the flash test application according to the following steps:

1. Connect one of the not yet tested flash chips to the Raspberry Pi. If there are no flash chips any more, then jump to step 10.
2. Power on the Raspberry Pi.
3. The Raspberry Pi boots up. After the U-Boot log was printed over the serial line, seL4 is started and with it comes the capdl-loader log messages.
4. Wait until you see "Starting NOR Flash test.". At this point the flash test application starts.
5. Detect actual flash size sub-test has the following output:

Detect flash size

```
INFO: /host/seos_sandbox/components/StorageServer/src/StorageServer.c:469: client 1: offset=0,
size=8388608INFO: /host/src/tests/test_storage/components/Tests/src/test_OS_Storage.c:128: Detecting
available memory size...INFO: /host/src/tests/test_storage/components/Tests/src/test_OS_Storage.c:
141: Testing memory size: 4096 Bytes (0 MiB).INFO: /host/src/tests/test_storage/components/Tests/src
/test_OS_Storage.c:141: Testing memory size: 8192 Bytes (0 MiB).INFO: /host/src/tests/test_storage
/components/Tests/src/test_OS_Storage.c:141: Testing memory size: 16384 Bytes (0 MiB).INFO: /host/src
/tests/test_storage/components/Tests/src/test_OS_Storage.c:141: Testing memory size: 32768 Bytes (0
MiB).INFO: /host/src/tests/test_storage/components/Tests/src/test_OS_Storage.c:141: Testing memory
size: 65536 Bytes (0 MiB)...INFO: /host/src/tests/test_storage/components/Tests/src/test_OS_Storage.
c:165: Detected memory size: ...
```

If the detected memory size corresponds to 8MiB, then the message "FLASH SIZE OK" is printed and continues at step 6, otherwise "FLASH SIZE WRONG" and jumps to step 7.

6. Check every single memory block:

Test every single memory block

```
INFO: /host/src/tests/test_storage/components/Tests/src/test_OS_Storage.c:174: Testing every memory
block...INFO: /host/src/tests/test_storage/components/Tests/src/test_OS_Storage.c:185: Testing block
0 of 2048INFO: /host/src/tests/test_storage/components/Tests/src/test_OS_Storage.c:185: Testing block
50 of 2048INFO: /host/src/tests/test_storage/components/Tests/src/test_OS_Storage.c:185: Testing
block 100 of 2048...INFO: /host/src/tests/test_storage/components/Tests/src/test_OS_Storage.c:210:
Functioning flash up to block ...
```

If all memory blocks pass the test, then the message "All blocks working" is printed, otherwise "Defect blocks".

7. In the end, the test application prints whether the Flash is okay ("FLASH OK.") or whether it is defect ("FLASH DEFECT.").
8. The test application informs the user when all the tests are done ("All tests done.").

9. Power off the Raspberry Pi and go back to step 1 again.
10. All flashes are tested. End.

Building and running demo_hello_world

- Use TRENTOS Handbook and follow instructions from chapter 2.5 Using TRENTOS-M on Raspberry Pi 3 B+
- Build hello world demo:

building hello world demo

```
sdk/scripts/open_trentos_build_env.sh sdk/build-system.sh sdk/demos/demo_hello_world/src rpi3 build-  
rpi3-Debug-demo_hello_world -DCMAKE_BUILD_TYPE=Debug
```

- Copy os_image.bin to SD card:

copy os_image.bin of hello world demo

```
sudo sd-mux-ctrl -e <sd_device_number> -ssudo mount /dev/<block_device_name> /mntcd  
<path_to_trentos_dir>/build-rpi3-Debug-demo_hello_world/imagesudo cp os_image.bin /mntsyncsudo  
umount /mntsudo sd-mux-ctrl -e <sd_device_number> -d
```

- You can now power on the Raspi and after booting the hello world demo should be executed